

Project: Smart Home Lighting System

Task 4.2D – Project Status

Project Recap

The Smart Home Lighting System project is an initiative to develop a robust, user-friendly, and scalable smart home lighting solution. The project addresses key limitations of existing systems, such as complex setup processes, interoperability issues, and a lack of truly adaptive and scalable control.

The ultimate vision is to create an intelligent system that not only allows for remote control but also intelligently adjusts lighting based on real-time environmental factors, occupancy, and user preferences.

The proposed solution is an IoT-enabled system that leverages sensors to gather environmental data, which is then processed by a cloud-based backend and used to intelligently control smart lights.

- The core processing logic is built using Node-RED flows for data aggregation and automation rules.
- The overall architecture is based on event-driven microservices deployed on Amazon Web Services (AWS) to ensure enhanced flexibility, maintainability, and scalability from the ground up.
- Key outcomes of this project will include a functional IoT-enabled prototype, a scalable AWS deployment, and comprehensive documentation.

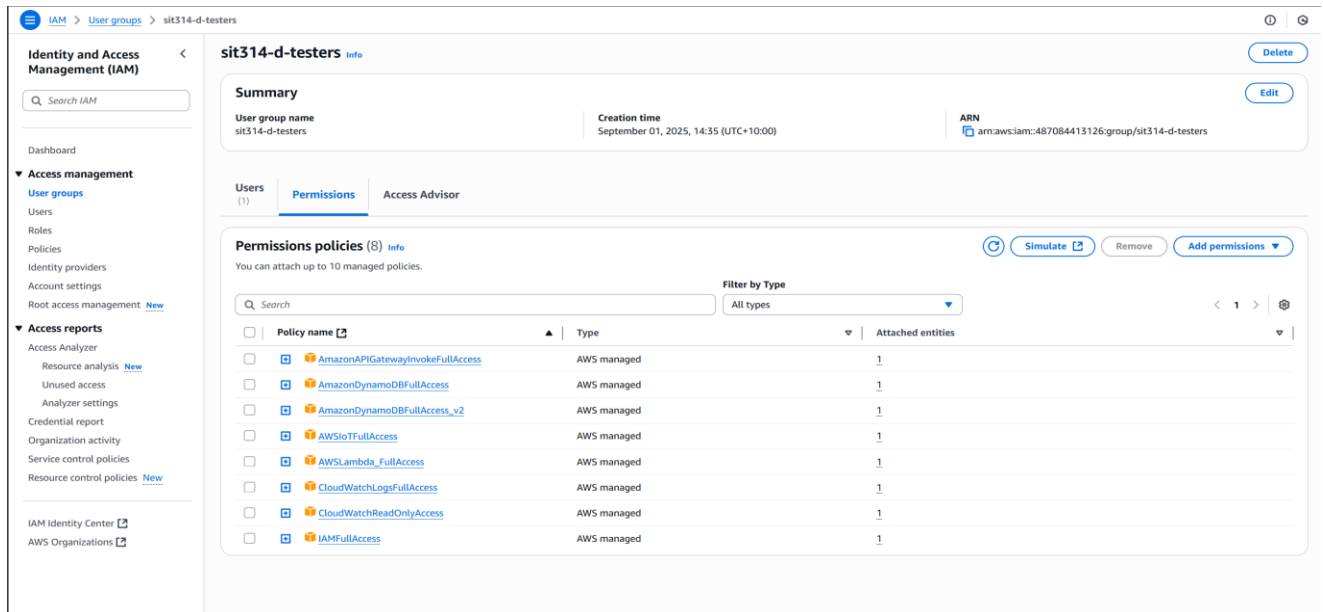
Achievements

1 – Foundational Architecture & Basic Data Flow

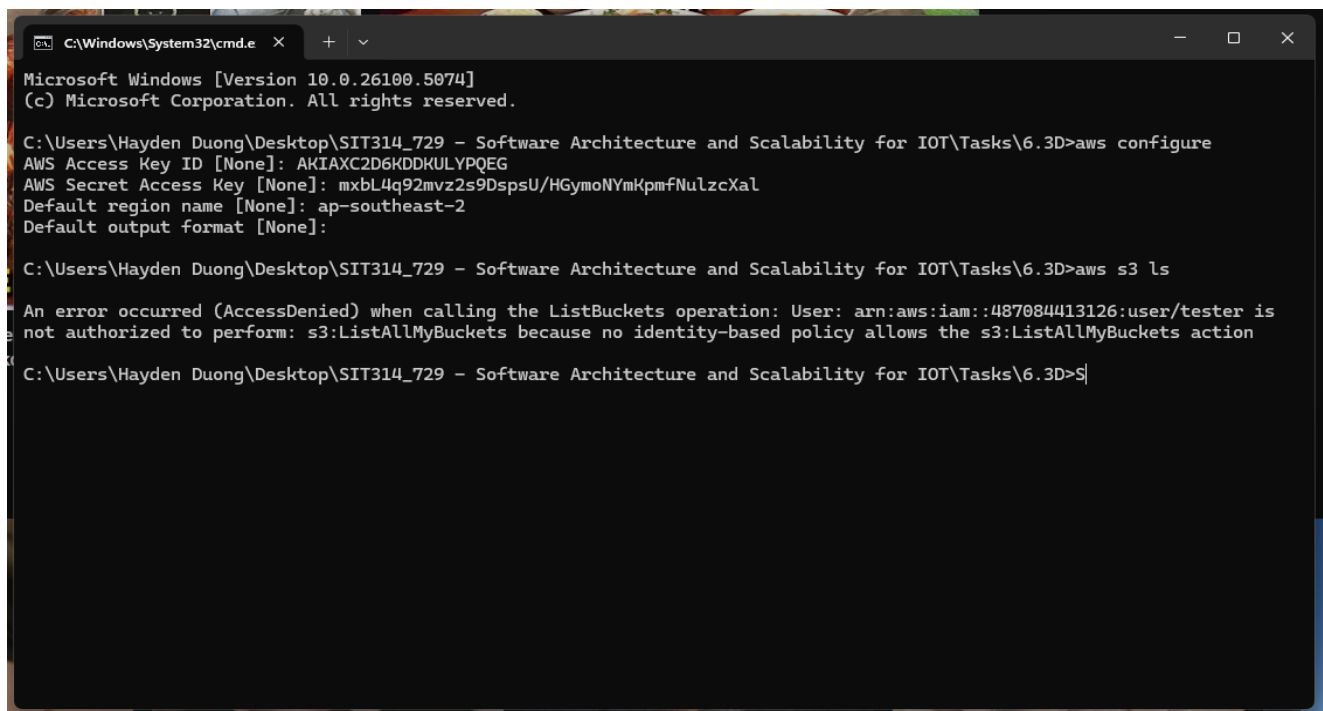
One of the main goals during the first phase was to focus on establishing a secure, modular, and scalable foundation on Amazon Web Services (AWS). This groundwork ensures the system can handle increasing loads and complexity as new features are added.

- **Secure AWS Infrastructure:** A dedicated IAM (Identity and Access Management) user was created with a least-privilege policy, ensuring secure and controlled

access to project resources. The AWS Command Line Interface (CLI) was configured to operate under this user, establishing a secure programmatic interface.

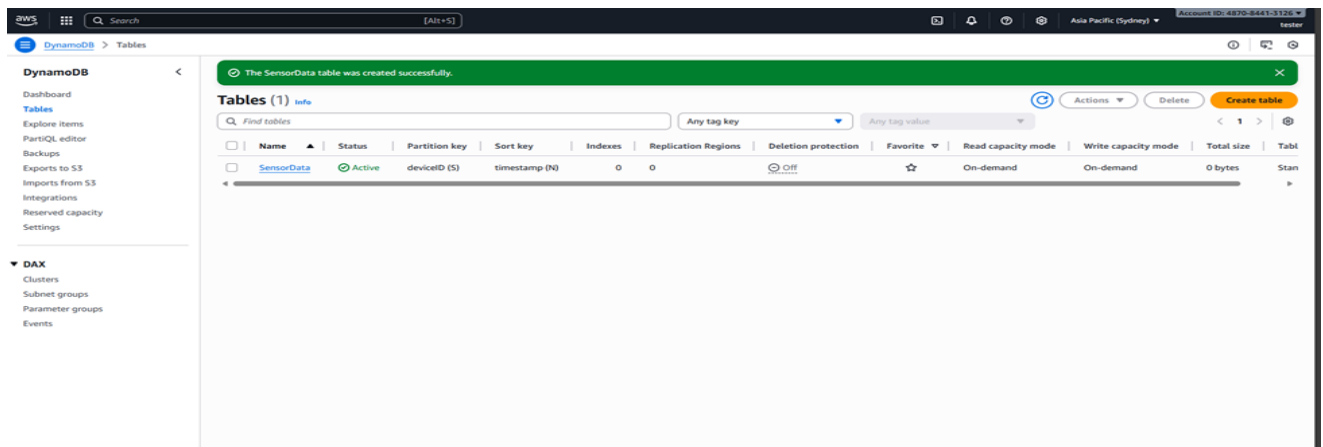


Picture 1 – IAM user account with limited permission policies



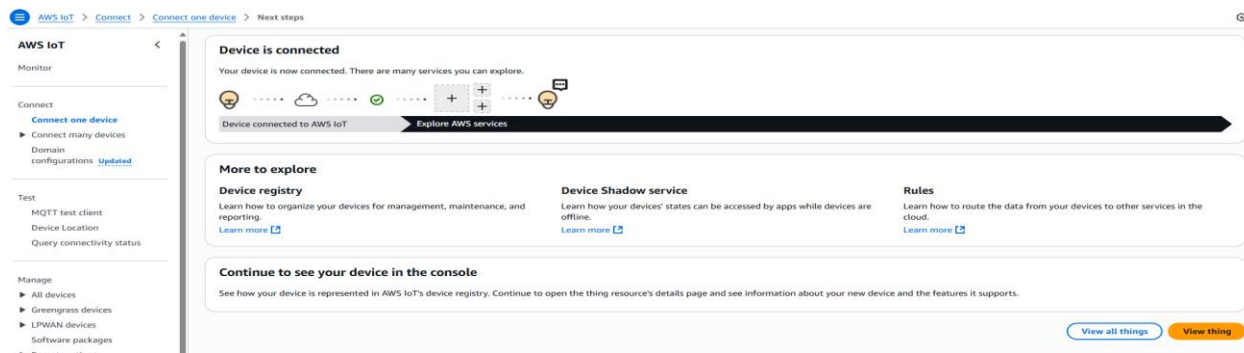
Picture 2 – Logged in as IAM user instead of root account & test its authorization.

- **System Architecture and Data Design:** The system was designed around microservice architecture, where each function (e.g., data processing, actuator control) is a self-contained service. This modular approach is critical for scalability, as it allows each component to be independently developed, deployed, and scaled. The data schema for the “SensorData” DynamoDB table was finalized with a time-series structure using:
 - “deviceId” as the partition key.
 - “timestamp” as the sorting key.

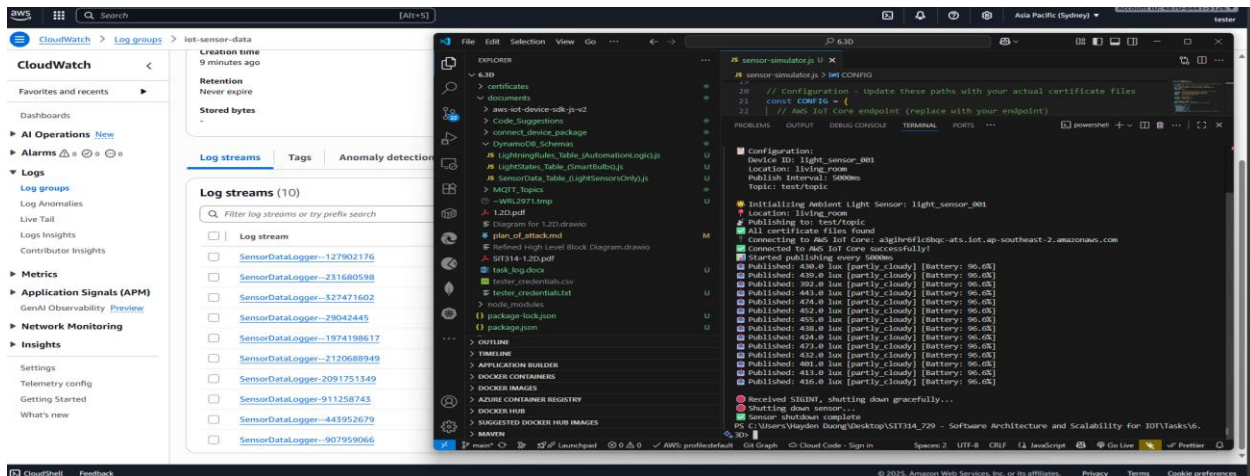


Picture 3 – Create “SensorData” Table on DynamoDB

- **Initial Data Pipeline:** A basic end-to-end data flow was successfully established. A simulated sensor was configured to send data via MQTT to **AWS IoT Core**. An IoT Rule then ingested this data, routing it to both **CloudWatch Logs** for monitoring and a **DynamoDB** table for storage.



Picture 4 – Successfully connected to AWS IoT

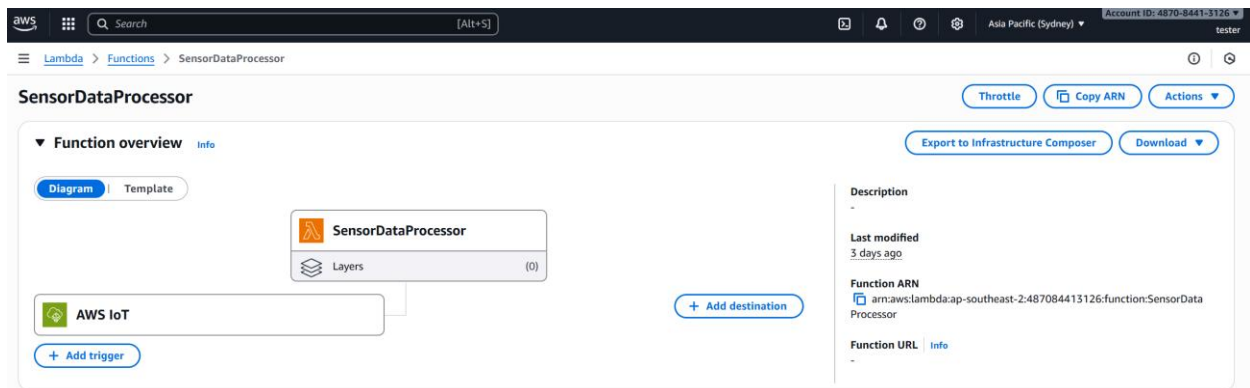


Picture 5 – Successfully displayed the sent payload on CloudWatch Logs.

2 – Core Logic, Automation, and Reliability

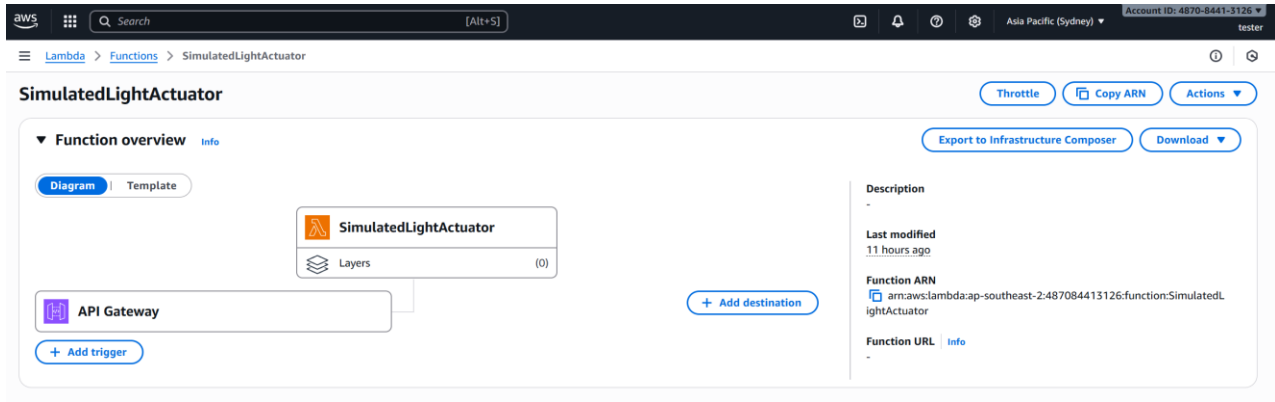
In the second half of the first phase, the focus was shifted from infrastructure to implementing the core logic and adding features that make the system intelligent and resilient.

- **Sensor Data Processing:** A “**SensorDataProcessor**” Lambda function was built to act as the primary intake for all sensor data. It parses incoming messages, handling different key formats and ensuring clean, structured data is written to the “SensorData” DynamoDB table.



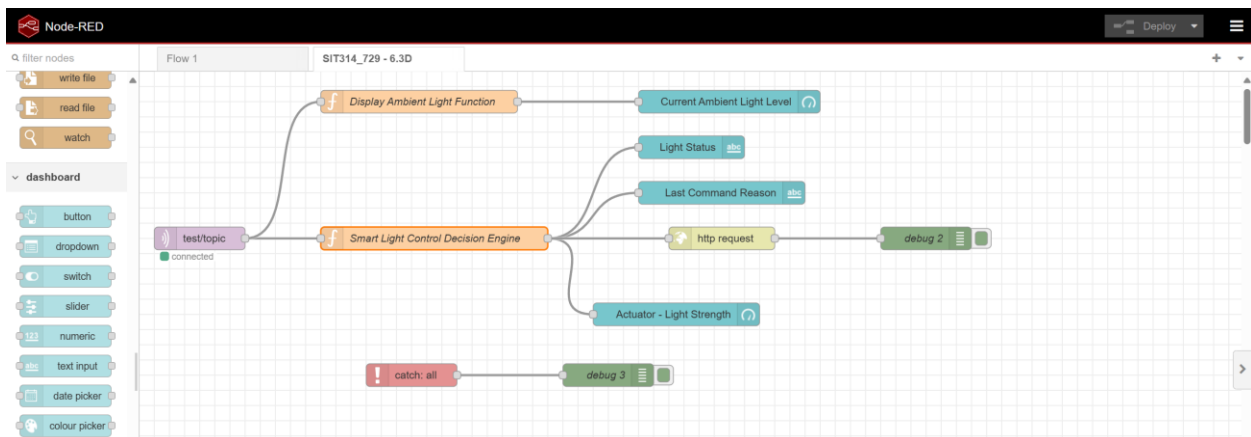
Picture 6 – Lambda function of “SensorDataProcessor”

- **Actuator Integration and Control:** The system's control mechanism was established using a “**SimulatedLightActuator**” Lambda function, exposed via an **API Gateway HTTP API**. This provides a scalable, stateless endpoint for receiving and processing light control commands.

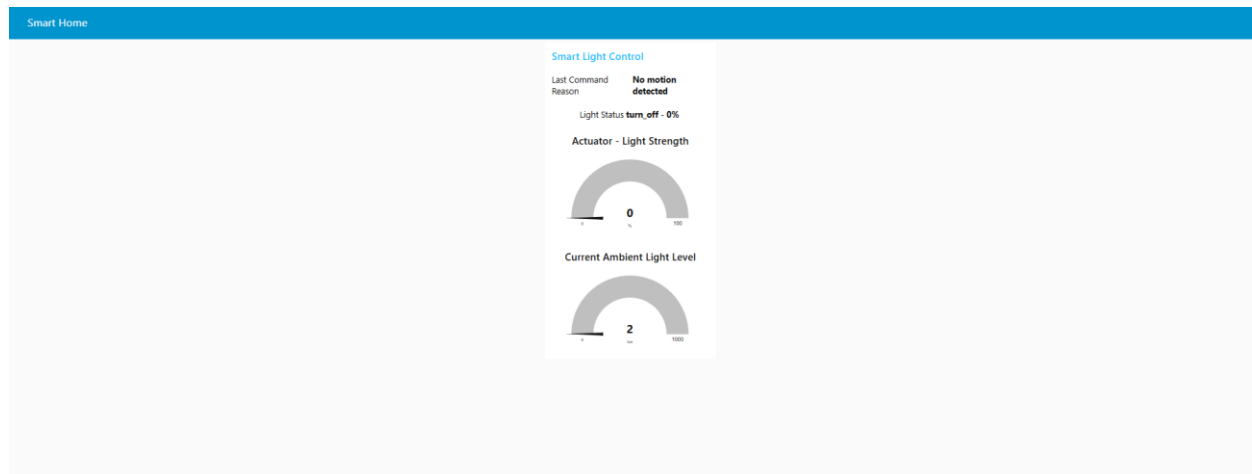


Picture 7 – Lambda function of “SimulatedLightActuator”

- **Decision Logic with Node-RED:** **Node-RED** was set up to serve as the central brain of the system. It subscribes to sensor data and applies a decision-making process based on predefined rules, such as:
 - **Motion and Ambient Light:** Turning a light on if motion is detected and the ambient light is below a specific threshold.
 - **Time-Based Policies:** Adjusting brightness levels throughout the day (e.g., 60% in the morning, 90% in the evening) to create a more realistic and energy-efficient experience.

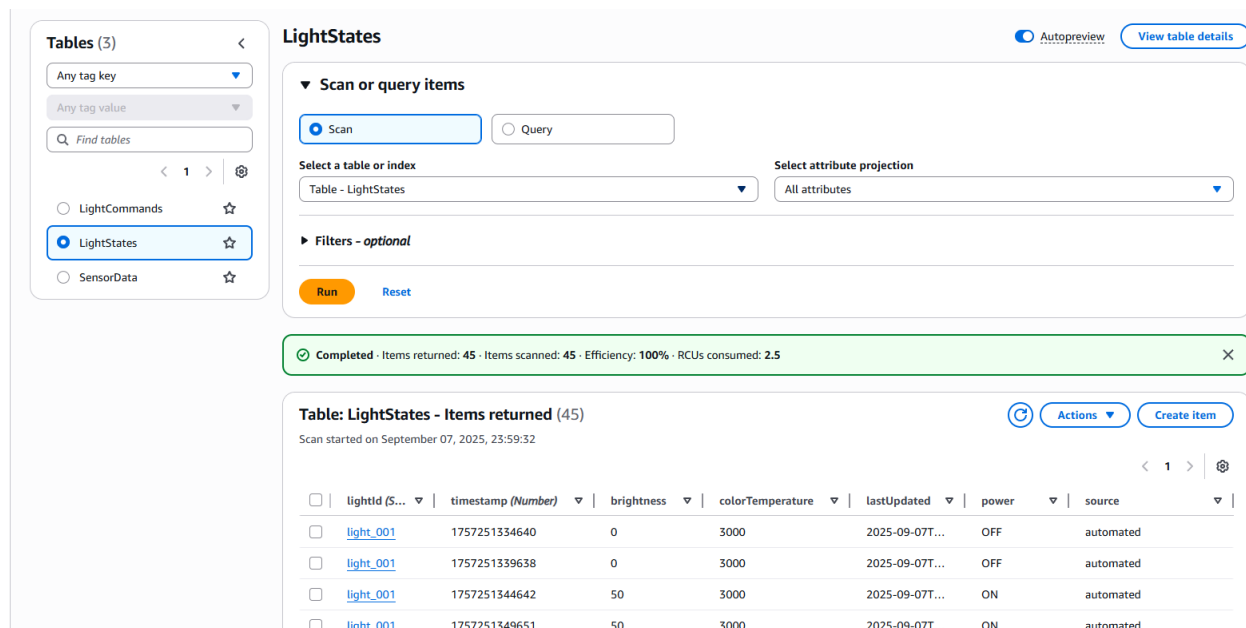


Picture 8 – Current node-RED setup of the project.



Picture 9 – Dashboard UI to display the light level of both the actuator & the ambient light in gauge format.

- **Enhanced Reliability and State Management:** A dual-table DynamoDB pattern was implemented to separate system intent from the actual state. The **LightCommands** table provides an immutable audit trail of every command sent, while the **LightStates** table stores the current, append-only state of each light. This design improves traceability and consistency.



Picture 10 – Created 2 additional tables, “LightCommands” & “LightStates”, on DynamoDB

- **Error Handling and Retries:** To increase system resilience, **try/catch** blocks were implemented in both Lambda functions and Node-RED flows. The Actuator Lambda now includes exponential backoff retries for DynamoDB writes, ensuring data persistence even during temporary service interruptions.

Next Steps – Preparation for Scalability

The foundational architecture from the first phase has been specifically designed to enable significant scalability, directly addressing a core project requirement. The next phase will focus on demonstrating this capability through testing and advanced monitoring.

- **Microservices and Serverless Components:** The use of **AWS Lambda** and **DynamoDB** provides native scalability.
 - Lambda functions automatically scale to handle thousands of concurrent requests without manual intervention, and DynamoDB's on-demand capacity mode ensures it can handle high-volume data writes and reads with consistent performance.
- **Stateless Components:** The **API Gateway** and the **Decision Engine** (Node-RED) are designed to be stateless, meaning they do not store session data.
 - This allows for seamless horizontal scaling, as any new instance can handle requests without needing to know the history of previous interactions.
- **Load Testing:** In the first half of this second phase, a simulated load testing script will be used to simulate thousands of devices concurrently sending messages.
 - This will rigorously test the system's ability to handle high throughput and confirm that the architecture holds up under stress.
- **Comprehensive Monitoring:** CloudWatch dashboards and alarms will be configured to monitor key performance indicators (KPIs) such as Lambda invocation rates, DynamoDB read/write capacity, and API latency.
 - This will provide real-time evidence of the system's scalability and help identify any performance bottlenecks.

By completing these steps, the project will not only have a functional IoT solution but also a demonstrable, data-driven case for its scalability and robustness, fulfilling the distinction task requirements.

References

A - Lambda function of “SensorDataProcessor”

```
import { DynamoDBClient, PutItemCommand } from "@aws-sdk/client-dynamodb";

const ddb = new DynamoDBClient({});

export const handler = async (event) => {
  console.log("event:", event);

  const msg = typeof event === "string" ? JSON.parse(event) : event;

  const deviceId = msg.deviceId ?? msg.sensorId ?? "unknown_device";
  const timestamp = Number(msg.timestamp ?? Date.now());
  const light = msg.ambientLight ?? msg.lightIntensity ?? 0;

  if (!deviceId || Number.isNaN(timestamp)) {
    console.log("Missing deviceId/timestamp", msg);
    return { statusCode: 400 };
  }

  const item = {
    deviceId: { S: deviceId },
    timestamp: { N: String(timestamp) },
    location: { S: msg.location ?? "unknown" },
    lightIntensity: { N: String(light) },
    weather: { S: msg.weather ?? "unknown" },
    batteryLevel: { N: String(msg.batteryLevel ?? 0) },
    quality: { S: msg.readingQuality ?? "unknown" },
  };

  await ddb.send(new PutItemCommand({
    TableName: "SensorData",
    Item: item
  }));

  return { statusCode: 200 };
};
```


B - Lambda function of “SimulatedLightActuator”

```
import { DynamoDBClient, PutItemCommand } from "@aws-sdk/client-dynamodb";

const ddb = new DynamoDBClient({});

// Retry configuration
const MAX_RETRIES = 3;
const RETRY_DELAY = 1000; // 1 second

// Retry function for DynamoDB operations
async function retryOperation(operation, operationName, maxRetries = MAX_RETRIES)
{
  for (let attempt = 1; attempt <= maxRetries; attempt++) {
    try {
      console.log(`${operationName} - Attempt ${attempt}/${maxRetries}`);
      return await operation();
    } catch (error) {
      console.error(`${operationName} - Attempt ${attempt} failed:`,
error.message);

      if (attempt === maxRetries) {
        throw error;
      }

      // Wait before retry (exponential backoff)
      const delay = RETRY_DELAY * Math.pow(2, attempt - 1);
      console.log(`${operationName} - Retrying in ${delay}ms...`);
      await new Promise(resolve => setTimeout(resolve, delay));
    }
  }
}

// Structured error logging
function logError(error, context) {
  console.error(JSON.stringify({
    timestamp: new Date().toISOString(),
    level: 'ERROR',
    function: 'SimulatedLightActuator',
    error: error.message,
    stack: error.stack,
    context: context
  }));
}
```

```

export const handler = async (event) => {
  console.log("Raw event:", JSON.stringify(event, null, 2));

  // Handle different input types
  let body;
  if (typeof event.body === 'string') {
    try {
      body = JSON.parse(event.body);
    } catch (error) {
      // If it's just a string like "ON" or "OFF", create a proper body
      body = {
        action: event.body === "ON" ? "turn_on" : "turn_off",
        brightness: event.body === "ON" ? 80 : 0,
        colorTemperature: 3000,
        reason: "Manual control from Node-RED"
      };
    }
  } else {
    body = event.body || {};
  }

  const pathParams = event.pathParameters || {};
  const lightId = pathParams.lightId || body.lightId || 'unknown_light';
  const action = body.action || 'set_state';
  const brightness = Number(body.brightness ?? 100);
  const colorTemperature = Number(body.colorTemperature ?? 3000);
  const reason = body.reason || 'No reason provided';

  console.log('Actuator request:', { lightId, action, brightness,
    colorTemperature, reason });

  try {
    // Simulated processing delay
    await new Promise(r => setTimeout(r, 400));

    // "Simulated" final state
    const state = {
      lightId,
      power: action === 'turn_off' ? 'OFF' : 'ON',
      brightness: action === 'turn_off' ? 0 : Math.max(0, Math.min(100,
brightness)),
      colorTemperature
    };
  }

  console.log('Actuator applied state:', state);

```

```

// Create command record for LightCommands table
const commandRecord = {
  commandId: { S: `cmd_${Date.now()}` },
  timestamp: { N: String(Date.now()) },
  lightId: { S: lightId },
  action: { S: action },
  brightness: { N: String(brightness) },
  colorTemperature: { N: String(colorTemperature) },
  reason: { S: reason },
  status: { S: 'executed' },
  sensorData: { S: JSON.stringify(body.sensorData || {}) }
};

// Create state record for LightStates table
const stateRecord = {
  lightId: { S: lightId },
  timestamp: { N: String(Date.now()) },
  power: { S: state.power },
  brightness: { N: String(state.brightness) },
  colorTemperature: { N: String(state.colorTemperature) },
  lastUpdated: { S: new Date().toISOString() },
  source: { S: "automated" }
};

// Store command in LightCommands table with retry logic
await retryOperation(
  () => ddb.send(new PutItemCommand({
    TableName: "LightCommands",
    Item: commandRecord
  })),
  "LightCommands PutItem"
);

console.log('Command logged to LightCommands table');

// Store state in LightStates table with retry logic
await retryOperation(
  () => ddb.send(new PutItemCommand({
    TableName: "LightStates",
    Item: stateRecord
  })),
  "LightStates PutItem"
);

```

```
console.log('State logged to LightStates table');

return {
  statusCode: 200,
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    ok: true,
    state,
    commandId: commandRecord.commandId.S
  })
};

} catch (error) {
  // Structured error logging
  logError(error, { lightId, action, brightness, colorTemperature, reason });

  return {
    statusCode: 500,
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      ok: false,
      error: error.message,
      retry: true // Add retry flag for client-side handling
    })
  };
}
};
```

C - Node-RED Smart Light Control Decision Engine

```
// Smart Light Control Decision Engine
const sensorData = msg.payload;

try {
  // Extract sensor values
  const lightLevel = sensorData.ambientLight || 0;
  const motion = Math.random() > 0.7; // Simulate motion in Node-RED
  const timestamp = sensorData.timestamp || new Date().toISOString();

  // Time-based brightness calculation
  const hour = new Date().getHours();
  let baseBrightness = 80; // Default

  if (hour >= 6 && hour < 12) {
    baseBrightness = 60; // Morning - softer
  } else if (hour >= 12 && hour < 18) {
    baseBrightness = 70; // Afternoon - moderate
  } else if (hour >= 18 && hour < 22) {
    baseBrightness = 90; // Evening - bright
  } else {
    baseBrightness = 50; // Night - dim
  }

  // Decision Logic
  let shouldTurnOn = false;
  let reason = "";

  // Rule 1: If motion detected AND light level is low, turn on
  if (motion && lightLevel < 200) {
    shouldTurnOn = true;
    reason = `Motion detected in low light (${lightLevel} lux)`;
  }

  // Rule 2: If no motion for 5+ minutes, turn off
  else if (!motion) {
    shouldTurnOn = false;
    reason = "No motion detected";
  }

  // Rule 3: If light level is high enough, turn off
  else if (lightLevel >= 70) {
    shouldTurnOn = false;
    reason = "Sufficient natural light";
  }
}
```

```
// Create command message with time-based brightness
const command = {
  lightId: "light_001",
  action: shouldTurnOn ? "turn_on" : "turn_off",
  brightness: shouldTurnOn ? baseBrightness : 0,
  colorTemperature: 3000,
  reason: reason,
  timestamp: timestamp,
  sensorData: sensorData,
  timeOfDay: hour >= 6 && hour < 18 ? "day" : "night",
  baseBrightness: baseBrightness
};

// Set the output
msg.payload = command;
msg.topic = "light_control";

return msg;
} catch (error) {
  // This will trigger the Catch node
  node.error("Error in Smart Light Control Engine: " + error.message, msg);
  return null;
}
```